LECTURE 09

Validations and Transformations

Backend Engineering from First Principles
Large-font study notes • API design • Data integrity • Security

Core principle
Validate and transform client input at the API boundary, before significant business logic or database operations.

1. Where validations and transformations fit

A simplified backend has three layers:

Layer	Main responsibility
Repository	Database connections and data operations: queries, inserts, updates and deletes. May work with relational databases, Redis or other storage.
Service	Business logic and application functionality. May call repositories, send notifications/emails, store data or call webhooks.
Controller	HTTP-facing layer. Receives client data, calls the service method and returns an HTTP response.

Why separate controller and service? The controller handles HTTP concerns; the service focuses on business functionality. This keeps transport-specific logic separate from business logic.

```
Client -> Route matching -> Controller entry point
| validate + transform input
v
Service layer (business logic)
|
v
Repository (database)
|
v
Service result -> Controller -> Client
```

The lecture places validation and transformation after route matching and before significant controller/service execution. This can be implemented as reusable middleware or utility functions driven by a schema.

2. What is validation?

Validation checks whether incoming data satisfies the structure and restrictions expected by an API before significant application work begins.

Input can arrive in a JSON request body, query parameters, path parameters or HTTP headers.

Example: a JSON field `name` is required, must be a string, and must contain between 5 and 100 characters.

A schema-driven pipeline can check field presence, data type, length/range/format constraints and relationships between fields.

3. Why validate at the API entry point?

Suppose an API expects a book name as text but receives:

```
{
"name": 0
}
```

Without validation, the request can pass through controller → service → repository. The database may reject the insert because the column expects text. Poor error handling may return **500 Internal Server Error**, even though the actual problem is invalid client input.

> With entry-point validation, detect the invalid value early and return a client error such as 400 Bad Request.

Benefits: protect data integrity, avoid unexpected downstream states, reduce unnecessary database work, provide clearer errors and make API behavior predictable.

4. Three common categories of validation

4.1 Syntactic validation

Checks whether input follows an expected format or structure. Examples: email format, phone number format (country code and digits), and date representation such as year-month-day.

> Syntactic validation asks: Is the input shaped correctly?

4.2 Semantic validation

Checks whether the value makes sense in the application's domain. Examples from the lecture: date of birth cannot be in the future; an age of 365 is not plausible in the example.

> Semantic validation asks: Does this value make sense?

4.3 Type validation

Checks whether each field has the expected data type: string, number, array, boolean or a structured object. Array elements can have their own type constraints.

> Type validation asks: Is this the expected data type?

Category	Checks	Example
Syntactic	Format / structure	Email, phone or date format
Semantic	Meaning / domain rule	Date of birth is not in the future
Type	Data type	Number field receives a number, not a string

5. What is transformation?

Transformation applies operations to incoming data to convert it into a format or structure expected by the server or service layer.

Query-parameter example

Consider `GET /bookmarks?page=2&limit=20`. The API wants numeric values, but query parameters arrive as strings by default:

```
page = "2"
limit = "20"
```

If the raw strings are validated as numbers, type validation fails. The server casts them:

```
"2" -> 2
"20" -> 20
```

Then it checks page greater than 0 and less than 500, and limit greater than 0 and less than 10,000. **Casting** converts a value from one data type to another.

Normalization examples

The lecture demonstrates lowercasing an email, adding a + prefix to a phone number, and converting a date into the format expected by the service layer.

```
Input email: SanJu@Example.COM
Output email: sanju@example.com

Input phone: 919876543210
Output phone: +919876543210
```

The purpose is consistent data that is convenient for downstream logic.

6. Validation and transformation pipeline

A reusable pipeline can read incoming data, cast/normalize values, validate required fields and types, check format and business rules, return structured errors when invalid, and pass clean normalized data to the service layer.

> The exact order depends on the implementation. Query strings, for example, may need casting before numeric range checks.

7. Advanced validation examples

Required fields and format

The lecture's API expects `email`, `phone` and `date`. An empty JSON body reports all three as required. A malformed email and numeric phone (when a string is expected) produce separate errors. After correcting them, the request succeeds.

Semantic constraints

An example expects `dateOfBirth` and `age`. A future birth date is rejected. An age of 430 is rejected by the example's maximum-age rule of 120.

Cross-field and conditional validation

The lecture uses `password`, `passwordConfirmation` and `married`: password must have at least 8 characters; confirmation must match; if married is true, a `partner` field is required; if false, partner is optional.

Nested type validation

An array field must be an array, and each element can also be validated. The example requires every array element to be a string.

8. Frontend vs backend validation

Frontend validation	Backend validation
Primarily improves user experience	Protects security and data integrity
Gives immediate feedback in a form	Enforces rules at the server boundary
May prevent unnecessary requests	Must validate every incoming request
Can be bypassed or absent in other clients	Must not trust the frontend

A backend may receive requests from web apps, mobile apps, Postman/Insomnia, other servers or scripts. Not every client uses the same UI or validation code. A user can also modify client-side code or send requests directly.

> Frontend validation is for user experience; backend validation is for security and data integrity. Backend validation is mandatory.

```
User fills form
|
v
Frontend validation
| invalid -> show immediate form error
| valid
v
Send API request
v
Backend validation + transformation
| invalid -> API validation error
| valid
v
Service / business logic -> Repository / database
```

9. HTTP status codes

Status	Meaning	Lecture context
200 OK	Request succeeded	Example request passes validation
400 Bad Request	Client sent invalid request data	Reject wrong type/format/rule before database work
500 Internal Server Error	Unexpected server-side failure	May occur if invalid input reaches deeper layers and causes an unhandled failure

10. Interview questions and answers

1. What is validation?
Checking incoming data against required structure, types, formats and rules.

2. Where should validation happen?
At the API entry point, after route matching and before significant business logic.

3. Why validate before database operations?
To protect data integrity, reject bad input early and avoid downstream failures.

4. What is syntactic validation?
Checking input format, such as email, phone or date representation.

5. What is semantic validation?
Checking whether a value makes sense in its domain, such as rejecting a future birth date.

6. What is type validation?
Checking that a field has the expected type, such as string, number, array or boolean.

7. What is transformation?
Converting or normalizing incoming data into the format expected by the server.

8. Why cast query parameters?
Query parameter values arrive as strings by default.

9. What is cross-field validation?
A rule comparing multiple fields, such as password confirmation matching password.

10. What is conditional validation?
A field becomes required depending on another field, such as partner when married is true.

11. Can frontend validation replace backend validation?
No. Frontend validation improves UX; backend validation is mandatory for security and integrity.

12. Why use a reusable pipeline?
It centralizes input rules and transformations across APIs.

13. What status code is appropriate for invalid client input?
Commonly 400 Bad Request.

11. Last-minute revision sheet

Repository = database access. **Service** = business logic. **Controller** = HTTP request/response.

Validate and transform after route matching, before significant logic. Validate body, query, path and header data.

Syntactic = correct format. **Semantic** = meaningful value. **Type** = correct data type.

Transformation = cast, normalize or convert data. Query parameters arrive as strings by default.

Cross-field: password and confirmation match. Conditional: partner required only when married is true.

Frontend validation = UX. Backend validation = security and data integrity; always required. Invalid client input commonly receives 400 Bad Request.

12. Practical API design checklist

■ Which request fields are required?

■ What type should each field have?

■ What format should each value follow?

■ What length/range limits apply?

■ Which values are semantically invalid?

■ Are there rules involving multiple fields?

■ Does any input need casting or normalization?

■ Are nested objects and array elements validated?

■ Are validation errors clear and safe to return?

■ Does the backend enforce all rules independently of the frontend?

End of Lecture 09 notes. These notes are based on the supplied transcript.